Ben-Gurion University of the Negev
Faculty of Computer and Information Science

Methods for Detecting Attack

Final Project

Submitted by

Natalie Mirelashvili
Faculty of Computer and Information Science
Ben-Gurion University of the Negev
E-mail: miralshv@post.bgu.ac.il

[complete others]

June 12, 2026

Abstract

Large language model (LLM) agents are increasingly augmented with tools, persistent memory, retrieval components, and the ability to communicate with other agents, which extends their capabilities but also broadens their attack surface. This is particularly true for multi-agent systems, where sensitive information can propagate through internal channels – messages between agents, tool arguments, memory writes, and logs – before, or even without, reaching the final user-facing output. This project proposes an AutoResearch-driven red-teaming methodology for agentic and multi-agent LLM systems, in which an LLM agent iteratively proposes attack variants, evaluates them against a target system on a fixed task, scores the outcome, and retains improving variants. We instantiate two independent optimization loops, one maximizing synthetic sensitive-information exposure and the other maximizing task-utility degradation and operational-cost amplification, and evaluate them across AgentDojo and three multi-agent frameworks: AutoGen, CrewAI, and LangGraph. By measuring both final-output and internal-channel leakage, utility degradation, cost amplification, and cross-framework transferability, we aim to determine whether automatically optimized attacks outperform random and manually designed baselines, and whether attacks discovered on one framework generalize to others.

1 Introduction

Large language model (LLM)-based agents are increasingly used for tasks that go beyond single-turn question answering: they call external tools and APIs, retrieve and incorporate documents, read and write persistent memory, and plan a sequence of actions toward a goal. In multi-agent systems, several such agents are composed into a single pipeline for example, a planner that decomposes the task, a retriever or tool-using agent that gathers external information, a worker agent that carries out the core task, and a reviewer or guard agent that checks the result before it is returned to the user. This decomposition improves modularity and, often, task performance, but it also creates internal communication channels – inter-agent messages, tool-call arguments, memory writes, and logs that are not directly visible to the end user.

Existing evaluations of prompt-injection robustness for LLM-integrated systems typically focus on whether the final, user-facing response leaks sensitive content or deviates from the intended task [1, 2]. In an agentic or multi-agent pipeline, however, a sensitive value can pass through, and be exposed at several internal points before any final answer is produced. An attacker who injects malicious content into a retrieved document, a tool's output, a task input, or a message exchanged between agents may cause a secret to surface in a tool argument, an inter-agent message, or a memory entry, even when the system's final response to the user appears unremarkable. Evaluating only the final

output can therefore substantially underestimate how exposed such a system really is, and may also miss attacks that degrade task performance or increase operational cost without leaking anything at all.

This project investigates that gap directly. We ask (i) whether an automated, iterative red-teaming process can discover attacks that exploit the internal channels of agentic and multi-agent systems more effectively than random or manually designed attacks, and (ii) whether final-output-only evaluation indeed misses leakage and degradation that an internal-channel-aware evaluation would catch. Rather than hand-crafting every attack variant, we adopt an AutoResearch-style loop, described below: an LLM agent proposes an attack variant, the variant is run against a target agentic system on a fixed task, the outcome is scored against a chosen metric, and the variant is kept or discarded before the next is proposed. We instantiate this loop twice, for two complementary attacker goals: maximizing the rate at which a synthetic canary secret is exposed through any channel, final output or internal, and degrading task utility and/or amplifying operational cost (extra tool calls, latency, tokens) without necessarily leaking a secret. We study both loops across four systems – AgentDojo, AutoGen, CrewAI, and LangGraph, configured with a shared abstract pipeline so that results are comparable across frameworks. Throughout, the attacker only injects content into untrusted channels (retrieved documents, tool outputs, task inputs, inter-agent messages). no real user data is involved, and all sensitive information is represented by synthetic canary secrets in sandboxed tasks.

1.1 Agentic and Multi-Agent LLM Systems

An agentic LLM system augments a base language model with the ability to call external tools or APIs, retrieve and incorporate external documents, read and write to a memory store, and plan over multiple steps rather than respond in a single turn. AgentDojo [3] is one such environment: it provides realistic tool-using tasks together with an established benchmark for prompt-injection attacks and defenses, and serves in this project as a single-agent, tool-using reference point.

Multi-agent frameworks compose several agents of this kind into a single pipeline. AutoGen [4] structures this composition as a conversation in which agents exchange messages and may invoke tools or request human input. CrewAI [5] assigns each agent an explicit role, goal, and set of tools, and lets agents delegate subtasks to one another. LangGraph [6] represents the pipeline as an explicit graph of nodes and edges with shared, inspectable state, which makes it convenient to trace exactly where in the pipeline an attack takes effect. In all three frameworks, the additional roles: planner, retriever/tool agent, worker, reviewer/guard communicate through messages, shared memory, or delegation calls that never appear in the final response shown to the user, but that attacker-controlled input may still be able to influence or read from. The threat model introduced later in this

paper targets exactly these channels.

1.2 AutoResearch-Style Optimization

AutoResearch [7] denotes an iterative research-automation pattern in which an LLM agent edits an experimental configuration, prompt, or piece of code, runs an evaluation, scores the outcome against a target metric, and keeps the change only if it improves that metric, otherwise discarding it and proposing a different change. The pattern was originally proposed to automate iterative improvement of training and evaluation pipelines, but the underlying generate-evaluate-score-keep/discard cycle does not depend on what is being optimized.

This project adapts that cycle to automated red teaming. Here, the LLM agent searches not for configurations that improve benign task performance, but for attack variants changes to the injected content, its placement within the pipeline, the targeted agent, and its phrasing. That increase a chosen attack-effectiveness metric against a fixed target system and task. This differs from recent reinforcement-learning-based approaches to automated prompt injection, which optimize universal adversarial suffixes through gradient-based or policy-gradient training over model weights [8, 9]. Because the AutoResearch loop operates on natural-language attack variants that are proposed and judged by an LLM agent rather than on model parameters, it requires no gradient access and can, in principle, be applied directly to closed, black-box agentic systems.

1.3 Contributions

This project makes the following contributions:

- We instantiate an AutoResearch-style red-teaming loop for agentic and multi-agent LLM systems, in which an LLM agent iteratively proposes, evaluates, and selects attack variants against a fixed target system and task.
- We define two independent optimization objectives for this loop: maximizing the total exposure rate of a synthetic canary secret across final-output and internal channels, and maximizing a weighted combination of task-utility drop and operational-cost amplification.
- We define an evaluation methodology that separately measures final-output leakage and internal-channel leakage (in tool arguments, inter-agent messages, memory writes, and logs), enabling a direct test of whether output-only evaluation underestimates real exposure.
- We apply this methodology to four systems spanning single-agent (AgentDojo) and multi-agent (AutoGen, CrewAI, LangGraph) architectures configured with a shared

abstract workflow, and study whether attacks optimized on one framework transfer to the others.

- We compare AutoResearch-optimized attacks against clean, random, and manually designed baselines, under a range of prompt-based, guard-agent, redaction, and tool-output-filtering defenses.

2 Related Work

2.1 Prompt injection and automated attack generation

Prompt injection attacks have become a major security concern for LLM-integrated systems because trusted instructions and untrusted content are often processed in the same model context [1, 2]. Indirect prompt injection showed that malicious instructions can be embedded in external content, such as web pages or documents, and later activated when an LLM application consumes that content [1]. PromptRobust [10] further showed that small perturbations at the character, word, sentence or semantic level can significantly affect model behavior. These results show that attack success depends on the model, task, prompt structure and location of attacker-controlled input.

Manual prompt injections often require substantial effort and can be brittle across models and application settings [11, 12] which motivated automated attack generation. Universal adversarial suffixes showed that optimization-based search can generate token sequences that transfer across aligned models, including black-box systems [11]. JudgeDeceiver extended this idea to LLM-as-a-Judge systems by searching for adversarial sequences that cause the judge to prefer a target response [12]. These studies show that effective attacks can be generated automatically rather than manually crafted, making malicious behavior increasingly difficult to identify.

Recent works extends automated attack generation from static optimization to reinforcement learning, which is especially relevant for multi-step and agentic settings. AutoInject [8] formulates prompt injection as an RL problem and learns universal, transferable suffixes while jointly optimizing attack success and benign-task utility. This shows that attackers can learn reusable injection strategies from reward signals instead of hand-engineering prompts. Slingshot [9] applies a related RL approach to agent-to-agent jailbreaking, where success is measured by verifiable tool execution rather than subjective textual compliance. Together, these works frame reinforcement learning as a general mechanism for automating adversarial discovery when attackers can exploit structured rewards and behavioral feedback.

2.2 Defenses for LLM-integrated systems

Existing defenses for LLM-integrated systems are commonly divided into soft and hard defenses [2, 13]. This taxonomy captures the distinction between mitigation techniques applied at inference time and defenses that modify the underlying system design.

Soft defenses. Soft defenses aim to mitigate prompt injection without modifying the underlying model or application architecture [2]. Liu et al. [2] benchmark a diverse set of soft defenses, ranging from prompt modifications to inference-time detection techniques. For indirect prompt injection, BIPIA [14] introduces boundary-awareness mechanisms and explicit reminders that encourage the model to treat retrieved content as data rather than executable instructions. These methods are practical and easy to deploy, but their effectiveness often decreases against adaptive attacks or when models fail to consistently follow prompt-level safeguards [2, 14].

Hard defenses. In contrast, hard defenses seek to address prompt injection at the architectural level by enforcing stronger trust boundaries, context separation, or input authenticity guarantees [15, 16]. The StruQ [16] framework separates system instructions and user data into structured channels and fine-tunes the model to follow only the instruction channel [15]. By construction, this approach treats prompt injection as a context-separation problem rather than solely a malicious-text detection problem. Shield adopts a broader system perspective by defining integrity, source identification, attack detectability and utility preservation as required safety properties for LLM-integrated applications. Survey studies place these mechanisms within a wider defense toolbox that also includes corpus cleaning, robustness optimization and safe instruction tuning [13].

Overall, recent work suggests a shift from prompt-level mitigation toward architectural defenses that provide stronger guarantees through context separation, authenticated data flows and safer-by-design interfaces [14, 15, 16].

2.3 Threat modeling and automation for LLM systems

As LLM-integrated applications introduce new attack surfaces, several works have adapted classical threat-modeling methodologies to capture risks that are unique to AI systems. Tete et al. [17] propose a threat-modeling framework that combines STRIDE, DREAD, and Shostack's four-question framework to analyze threats such as prompt injection, data poisoning, jailbreaking and information disclosure. Building on this direction, Gadade et al. [18] argue that the dynamic and adaptive nature of LLM systems requires automated threat-modeling capabilities. Their framework integrates machine learning, NLP, behavioral profiling and anomaly detection to automatically identify, assess and prioritize security risks in LLM deployments.

STRIDE GPT [19] explores the automation of the threat-modeling process by gen-

erating threat models, attack trees, mitigations and security test cases from system descriptions or source code context. By reducing the expertise required to perform threat analysis, such systems aim to make security assessment more accessible and scalable.

Xu et al. [20] demonstrated that formal threat models can be transformed into attack paths and executable security tests, enabling systematic validation of security requirements. Similarly, Tuma et al. [21] showed that security design flaws can be detected automatically by applying model-query patterns to annotated Data Flow Diagrams. The Meta Attack Language further extends this direction by providing domain-specific languages for reusable threat modeling and attack simulation [22]. Collectively, these works illustrate a broader shift from manually constructed threat models toward automated frameworks that can generate, analyze, and operationalize security knowledge throughout the system lifecycle.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pp. 79–90, 2023.

[2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, (Philadelphia, PA), pp. 1831–1847, USENIX Association, Aug. 2024.

[3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents," *Advances in Neural Information Processing Systems*, vol. 37, pp. 82895–82920, 2024.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," 2023.

[5] CrewAIInc, "Crewai: Framework for orchestrating role-playing, autonomous ai agents." `https://github.com/crewAIInc/crewAI`, Nov. 2023.

[6] LangChain AI, "Langgraph: Building stateful, multi-actor applications with llms." `https://github.com/langchain-ai/langgraph`, Jan. 2024.

[7] A. Karpathy, "Autoresearch: A bounded, public loop for automated machine learning experimentation." `https://github.com/karpathy/autoresearch`, 2026.

[8] X. Chen, J. Zhang, and F. Tramer, "Learning to inject: Automated prompt injection via reinforcement learning," *arXiv preprint arXiv:2602.05746*, 2026.

[9] S. Nellessen and T. Kachman, "David vs. goliath: Verifiable agent-to-agent jailbreaking via reinforcement learning," *arXiv preprint arXiv:2602.02395*, 2026.

[10] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, "Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts," 2023.

[11] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023.

[12] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to llm-as-a-judge," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, (New York, NY, USA), pp. 660–674, Association for Computing Machinery, 2024.

[13] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (llm) security and privacy: The good, the bad, and the ugly," *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.

[14] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '25, Association for Computing Machinery, 2025.

[15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "Struq: Defending against prompt injection with structured queries," 2024. To appear at USENIX Security Symposium 2025.

[16] F. Jiang, Z. Xu, L. Niu, B. Wang, J. Jia, B. Li, and R. Poovendran, "Identifying and mitigating vulnerabilities in llm-integrated applications," 2023.

[17] S. B. Tete, "Threat modelling and risk analysis for large language model (llm)-powered applications," 2024.

[18] B. Gadade and G. A. Patil, "A threat modeling framework for llm system integration leveraging nlp and machine learning," *Communications on Applied Nonlinear Analysis*, vol. 32, no. 10s, 2025.

[19] M. Adams and P. Raghavan, "Se radio 648: Matthew adams on ai threat modeling and stride gpt." Software Engineering Radio podcast, 2024.

[20] D. Xu, M. Tu, M. Sanford, L. Thomas, D. Woodraska, and W. Xu, "Automated security test generation with formal threat models," *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 4, pp. 526–540, 2012.

[21] K. Tuma, L. Sion, R. Scandariato, and K. Yskout, "Automating the early detection of security design flaws," in *Proceedings of the ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems*, MODELS '20, pp. 332–342, Association for Computing Machinery, 2020.

[22] P. Johnson, R. Lagerström, and M. Ekstedt, "A meta language for threat modeling and attack simulations," in *Proceedings of the 13th International Conference on Availability, Reliability and Security*, ARES 2018, pp. 38:1–38:8, Association for Computing Machinery, 2018.